

What we have built, and how it compares

A review of three external reference points against the Snap Apps document engine — what is running today, what is planned, and the three things this comparison changes.

DATE	STATUS	AUDIENCE	SOURCE DOCS
12 September 2026	Review draft	Engineering & non-technical	docs/OCR.md, docs/AI.md, bench/results

The short version. Three things were examined: a public benchmark comparing three ways of serving the same small AI model; a teaching demo showing how compressing a model changes its speed and accuracy; and a scan of Hugging Face, the main public library of open AI models, for anything built to read Australian documents.

None of the three is a commercial competitor. All three are useful, because each one does something we have written down as important and not yet finished.

The scan produced the one finding that changes a decision: **the two open models our own architecture document names as the strongest available are licensed in a way that forbids us shipping them to a customer.** That needs to be recorded before anyone plans around them.

01 Why this review exists

Snap Apps reads Australian receipts and invoices and turns them into accounting records. The part that does the reading is called the **document engine**. Its architecture is written down in `docs/OCR.md`, a long and deliberately honest document whose competitive section opens with a table of how well rival systems perform — and then admits, in bold, that *every number in that table is someone else's*.

Three links were put forward for comparison. This document answers the question properly: what are they, what do we have, what is planned, and where does the comparison actually bite.

IN PLAIN TERMS

Reading a receipt with a computer sounds simple and is not. The difficulty is not reading the letters — it is knowing, afterwards, whether you read them correctly. A system that is confidently wrong about a tax figure is worse than one that says "I could not read this." Almost everything below is about that distinction.

The three references at a glance

REFERENCE	WHAT IT IS	WHAT IT ACTUALLY DOES	RELATIONSHIP TO US
sentiment.net "Experiment 001"	A public benchmark, not a product	Takes one sentence and one fixed AI model, then runs it through three different software stacks on identical hardware to see which serves it fastest.	Not a competitor. A method worth copying.
github.com/azlanabas/utmcslmv	A university teaching demo	Runs one small language model at three compression levels side by side so students can see the trade-off between model size, speed and accuracy.	Not a competitor. Fills a measurement gap we have.
Hugging Face search "OCR for documents, Australia"	The main public library of open AI models	Shows what the world has already published and made free to use.	Tells us what to build and what to borrow .

NOTE ON CATEGORY

The first two are small personal or academic projects — one has two commits and no stars. Nothing below treats them as rivals. They are treated as evidence, because a small project that has actually measured something outranks a large document that has not.

02 Where the Snap Apps engine stands today

Before comparing anything, an honest statement of our own position. The engine has three layers, and they are at three very different stages of completion.

CAPABILITY	STATUS	WHAT EXISTS IN THE CODEBASE	WHAT THAT MEANS IN PRACTICE
Reading the document	LIVE	A vision-language model is handed the whole page and asked for structured data. Default <code>gemma4:31b</code> ; a router escalates through four models on failure.	Works. But the model never reports <i>where</i> on the page it found a number — it goes straight from picture to answer.
Checking the reading	LIVE	Arithmetic, date and ABN checks in <code>validators.ts</code> , including the modulus-89 ABN checksum and a dual-convention line-total check.	This is our strongest asset and it is fully built. Deterministic code, not a model, decides whether a human must look.
Audit trail & replay	LIVE	Migration 0009: re-extraction updates in place, both runs survive, human corrections are locked, documents behind a posted transaction are frozen.	A past result can be reproduced and every correction is traceable. Rare in this field.
Data isolation	LIVE	Row-level security in the database; the background worker itself goes through it rather than around it.	One tenant cannot read another's documents, enforced by the database, not by application discipline.
Our own OCR stage	SHADOW	<code>packages/docai</code> plus a Python service wrapping PP-OCRv5. Built, tested, and wired in — but only in shadow mode.	It runs <i>alongside</i> the real result and never affects it. If the switch is unset, it is not called at all.
Location of every value	SHADOW	<code>grounding.ts</code> attaches coordinates to extracted fields — advisory only.	Cannot yet be relied on, because the stage that produces coordinates is not the primary reader.
Deployment restrictions	LIVE	<code>registry.ts</code> models residency as a <i>type</i> . An air-gapped profile does not forbid external engines — it removes them from the list.	A misconfigured flag cannot send Australian tax records offshore, because the option does not exist in that build.
Calibrated confidence	GAP	The engine self-reports <code>calibrated: false</code> . The cutoff of 0.8 is labelled in code as a placeholder.	When the system says it is 95% sure, that number does not yet mean anything measurable.
Evaluation corpus	GAP	Four documents, all synthetic, rendered from HTML templates in the <code>bench</code> folder. The manifest says so explicitly.	Good enough to exercise the harness. Not good enough to claim accuracy on real Australian paperwork.

Status as of 12 September 2026. "Shadow" means the code runs but its output is stored separately and never changes what the user sees.

The single most important line in this table: **the OCR stage is built but not switched on**. Every stored layout record from the most recent run reads `ppocr-v5 failed: docai-engine sidecar not reachable`. Shadow mode that never executes collects no evidence, and the calibration work planned next depends entirely on that evidence.

What we have actually measured

As of 11 September 2026 the competitive table is no longer entirely borrowed. Two benchmark runs exist, on our own harness, on our own documents.

4 × 4 × 5	0.875	0.0000	n/a		
documents × models × repeated runs in the extraction benchmark	text-detection recall of the OCR stage — 7 of 8 regions found	character error rate, ignoring spacing — the characters are read perfectly	confidence-vs-error correlation — not enough data to compute		
MODEL	EASY RECEIPT	HARD RECEIPT	NO-ABN RECEIPT	2-PAGE INVOICE	FIELD OUTCOMES
minimax-m3	4.2 s	5.7 s	3.4 s	13.0 s	Clean — no wrong, no misses, both abstentions correct
kimi-k3	5.9 s	7.7 s	5.4 s	28.6 s	Clean — same accuracy, roughly twice the time
gemma4:31b (default)	2.4 s	2.9 s	1.8 s	4.1 s	Fastest — but 2 wrong and 1 miss across the set
qwen3.5:397b	21.2 s	27.3 s	20.4 s	50.2 s	5–10× slower for no accuracy gain over minimax-m3

Median seconds over 5 repeated runs, 11 September 2026. Documents are synthetic. "Abstention correct" means the document genuinely had no ABN and the model correctly returned nothing rather than inventing one.

WHAT THIS TABLE IS WORTH

It is real, it is ours, and it is small. Four synthetic documents cannot tell us how the system behaves on a crumpled thermal docket photographed in a car park. That corpus is the next thing to build, and the architecture document already says so.

It does, however, settle one live question: the largest model is not the best one. **qwen3.5:397b costs five to ten times the latency of minimax-m3 and reads no better.**

What is planned

PHASE	EFFORT	STATE	WHAT IT DELIVERS
0 — Stop lying to the user	1–2 weeks	DONE	Multi-page capture fixed, PDF intake accepted, evaluation harness built. Gate passed 11 Sep 2026 — and found six defects no unit test had caught.
1 — Reading stage	4–6 weeks	NEXT	Promote the OCR stage from shadow to primary. The AI model is demoted from <i>the reader</i> to <i>the fallback for hard patches</i> . Calibrated confidence.
1b — Device tier	2–3 weeks	PLANNED	The same reading stage running on a phone, offline.
2 — Tables	4–6 weeks	PLANNED	Business documents with real table structure.
3 — Figures	3–4 weeks	PLANNED	Charts and their underlying series.
4 — Forms	4–6 weeks	PLANNED	Filled-in forms and handwriting.
5 — Portability	ongoing	PLANNED	On-premises packaging, and the corrections flywheel: every human fix becomes training data.

The four-tier routing model

Both planned and partly live. The principle is **cheapest capable engine first, escalate only on failure** — implemented today in `pipeline.ts`.

TIER	WHAT READS THE PAGE	RUNS WHERE	COST PER PAGE	STATE
T0	Text already embedded in a PDF	Anywhere	Free, and exact	LIVE
T1	Our own OCR stage (PP-OCRv5)	Ordinary CPU, offline, phone	Near zero	SHADOW
T2	Open AI model on our own GPU	Our cloud, customer's cloud, air-gapped	GPU-seconds	PLANNED
T3	Frontier AI model (Claude, Sydney region)	Cloud deployments only	~\$6–11 per 1,000 pages	LIVE

Today almost everything goes to T3 because T1 is not switched on. The economics of the whole plan depend on reversing that: T1 handling most of the page, T3 seeing only the genuinely hard 5–15%.

03 Reference one — the sentiment benchmark

sentiment.net1io.space is an experiment, presented as one. It has sections headed *Apparatus*, *Method*, *Observation* and *Control*. One sentence goes in; one fixed model classifies it as positive or negative; the only thing that changes is the software stack serving it.

HELD CONSTANT	DETAIL
Model	distilbert-base-uncased-finetuned-sst-2-english — a small, standard sentiment classifier
Hardware	One rented server, 2 CPU cores
Interface	POST /analyze — identical for all three stacks

VARIABLE	STACK	WHAT IS BEING TESTED
Stack A	FastAPI + Python (PyTorch)	The conventional approach — and the one our own OCR service uses
Stack B	Rust / Axum bridging to Python	A fast front end still calling Python for inference
Stack C	Rust / Axum + rust-bert (native)	No Python at all in the serving path

Comparison

DIMENSION	THEIR EXPERIMENT	SNAP APPS — CURRENT	SNAP APPS — PLANNED
Question asked	Which serving stack is fastest?	Which AI model reads a receipt best?	Unchanged, plus: is our own OCR stage good enough to take over?
Scoring	Latency	Per-field, with <i>correct abstention</i> as a distinct success and <i>hallucination</i> as worse than a miss	Adds calibration: does "95% sure" mean right 95% of the time?
Repeats	Live, per request	5 repeats, medians reported	Same, on real documents
Missing-engine handling	n/a	Reports "not run" with a reason; never invents a score	Unchanged
Serving layer tested	Yes — it is the whole point	Never	Not in the plan

Our harness is the more sophisticated instrument. Their *discipline* is the thing worth taking: **hold everything constant, change exactly one thing, publish the number you measured**. On the one dimension they chose, they have data and we have none.

Why the serving layer is not an idle question for us

Our OCR service is Python and FastAPI — their Stack A. It currently runs with a CPU accelerator switched off, because PaddlePaddle 3.3.1 throws an error on both detection models with it enabled. The workaround is documented and correct. It also means the stage is running slower than its ceiling, and we have never measured what a different serving architecture would recover.

IN PLAIN TERMS

Two parts of the system could be responsible for slowness: the AI itself, or the plumbing around it. We have measured the AI. We have never measured the plumbing. This experiment is a template for doing that in about a day.

04 Reference two — the quantization demo

`utmcs_llmv` is a university teaching tool, deployed 6 September 2026, with two commits and no stars. It demonstrates **quantization**: compressing an AI model so it is smaller and faster, at some cost to accuracy. It runs one model — Llama 3.2 1B — at three compression levels and lets a student watch the trade-off.

IN PLAIN TERMS

Quantization is like saving a photo as a smaller JPEG. The file shrinks and loads faster; at some point the picture starts to look wrong. The skill is knowing where that point is. For AI models it matters enormously, because the difference between compression levels can be the difference between needing an expensive GPU and not.

Their architecture, and ours

CONCERN	UTMCS_LLMV	SNAP APPS
Front end	Next.js, TypeScript	Next.js web, React Native mobile
Back end	Rust / Axum	Node / Fastify, plus a Python OCR service
Store	SQLite, append-only	PostgreSQL with row-level security, append-only edit history
Where AI runs	A Mac on the desk, reached over a private network	Hosted models today; our own CPU stage in shadow
Separation principle	The web server never runs a model — it stores results and proxies when the Mac is awake	The same idea, expressed as a <i>type</i> : a deployment profile filters the engine list, so a forbidden engine is absent rather than disabled

WHERE WE ARE AHEAD

Ours is the stronger design. Their separation is a network arrangement that can be misconfigured; ours is enforced in code — `permitted()` removes an engine from the list entirely. Their store has no audit trail, no locked human corrections, no tenant isolation. Ours has all three, tested against a real database.

WHERE THEY ARE AHEAD

Theirs is **running**. Ours is behind an unset environment variable, and every stored record says the service was unreachable. A split-brain design that has never completed a run has not been proven — it has been described.

The gap this exposes

Our tier table names "an open AI model on our own GPU" as tier 2 and never pins which model, at which compression. GPU cost is the largest number in the plan's cost section. Meanwhile, in our engine registry, the field recording how

long each engine takes reads:

```
medianSeconds: number | null; /* Measured median seconds per page. null until someone measures it. */
```

QUESTION	THEM	US — CURRENT	US — PLANNED
Effect of compression on speed	Measured, 3 levels	Never measured	Not in the plan
Effect of compression on accuracy	Measured	Never measured	Not in the plan
Cost per page at tier 2	n/a	Estimated, not measured	Still an estimate
Per-engine timing in the registry	n/a	null	To be filled by the harness

A two-commit teaching repository has plotted the curve that our cost model assumes and has never drawn. The harness to draw it already exists in our codebase. This is the cheapest high-value measurement available to us.

05 Reference three — the public model library

Hugging Face is where the world publishes open AI models. The search page is built in a way that hides results from automated retrieval, so the site's data interface was queried directly instead. Two findings, and the second changes a decision.

Finding one — nobody has built this for Australia

SEARCH	BEST RESULT	WHAT IT ACTUALLY IS
"australian" — models	isaacus/open-australian-legal-gpt2	Legal text generation. Also returned: text-to-speech voices, and a dog-breed image model.
"australian" — datasets	ATO-Australian-Tax-Rulings	Tax <i>rulings as text</i> . Useful, but not documents to be read.
"receipt" — models	SZL-Forge-1.5B-ReceiptAgent	1,970 downloads. The rest of the top ten are largely Japanese receipt models, one German.

No model anywhere in the public library is trained to read Australian receipts, invoices or tax documents.

Meanwhile the general "read any page" category is saturated:

MODEL	DOWNLOADS	LICENCE	CAN WE SHIP IT TO A CUSTOMER?
datalab-to/chandra-ocr-2	2,908,785	OpenRAIL	No — fails our licence floor
baidu/Unlimited-OCR	2,700,194	MIT	Yes
deepseek-ai/DeepSeek-OCR	2,400,151	MIT	Yes
zai-org/GLM-OCR	1,893,869	MIT	Yes
datalab-to/surya-ocr-2	1,206,466	OpenRAIL	No — fails our licence floor
PaddlePaddle/PP-OCRv5	763,379	Apache-2.0	Yes — this is what we already use
stepfun-ai/GOT-OCR2_0	670,426	Apache-2.0	Yes
dots-studio/dots.ocr	368,761	MIT	Yes
nanonets/Nanonets-OCR-s	351,066	none stated	No — unstated licence is an unusable licence

Licences verified against the Hugging Face model index, 12 September 2026. Our rule allows only Apache-2.0 or MIT weights in a build handed to a customer.

Finding two — the two strongest models are ones we cannot ship

Our architecture document singles out two open systems by name: **Chandra 2** as the "best single published number", and **Surya 2** as the "best cost/accuracy of the small models". Both are OpenRAIL-licensed. Our own licence rule — written into code as a function that scans the engine list and returns violations — would reject both from any package sent to a customer.

IN PLAIN TERMS

A licence is the legal permission attached to a model. Some open licences let you use a model freely but not redistribute it inside a product you sell. OpenRAIL is that kind. For a consumer app that calls a model over the internet it rarely matters. For an enterprise customer who wants the system installed inside their own building, it decides whether the product can exist at all.

Our own document anticipated this in general — it warns that discovering a licence problem during a customer's procurement review is "a bad day that CI can have instead." It just never named which two models the warning applies to.

The comparison table in `docs/OCR.md` §2 currently reads as a menu of options. Two of its headline entries are not on the menu. A licence column should be added, so nobody plans around a model we cannot deliver.

What the saturation tells us

STRATEGIC QUESTION	WHAT THE LIBRARY SHOWS	CONSEQUENCE FOR US
Should we compete on general page reading?	Four models with over 1.8 million downloads each, heavily funded, measured to two decimals	No. We would start a year behind. Our plan already says so; the download counts confirm it.
Is there an Australian model to borrow?	Nothing. Not one.	The shelf is empty because only someone with the data can fill it.
Do we have that data?	n/a	Yes. Every human correction in the review screen has been stored since migration 0004 as a matched triple: the image, what the machine read, what the person changed it to.
Is the tier-2 slot fillable under our licence rule?	DeepSeek-OCR, GLM-OCR and dots.ocr are all MIT and all widely used	Yes. Losing Chandra and Surya is a narrowing, not a blockage.

06 The comparison in one table

The dimensions that matter to a finance customer, across all four systems. Two of the four are small side projects and are not expected to score here — they are included to show what a research demo does and does not have to care about.

CAPABILITY	SENTIMENT EXPERIMENT	UTMCS_LLMV	OPEN OCR MODELS	SNAP APPS TODAY	SNAP APPS PLANNED
Reads Australian tax documents	No	No	No	Yes	Yes, wider
Says where on the page a value came from	n/a	n/a	Block level at best	Shadow only	Required on every value
Confidence that means something	No	No	Not reported	Not calibrated	Calibrated and measured
Can say "I could not read this"	No	No	Largely absent	Yes, scored as a success	Yes
Catches its own mistakes	No	No	No	Arithmetic + ABN checksum	Plus engine disagreement
Reproduce a past result exactly	No	Results stored	No	Yes, versioned runs	Yes
Audit trail of human corrections	No	No	No	Yes, with locking	Yes, feeds training
Can run with no internet	Locally, yes	Private network	Most can	Built, not switched on	Yes, enforced by type
Data residency guaranteed	No concept	No concept	No concept	Engine registry	Same
Tenant data isolation	No	No	n/a	Database-enforced	Same
Serving stack measured	Yes — the whole point	Partly	n/a	Never	Not in the plan
Model compression measured	No	Yes, 3 levels	n/a	Never	Not in the plan
Accuracy on real captured documents	n/a	n/a	Public benchmarks	Synthetic only	Real gold sets

"n/a" means the dimension does not apply to that kind of system. Red entries against the two side projects are not criticism — a sentiment benchmark has no reason to have an audit trail.

Read the last three rows together. **Every gap we have that these references do not is a measurement gap, not a capability gap.** We have built more than either of them; we have measured less of it than either of them has measured of theirs.

07 What this changes

Four actions, in the order they should be taken. Two are hours of work; two are days.

	ACTION	EFFORT	WHY
1	Start the OCR service and complete one real shadow run	Hours	Every stored layout says the service was unreachable. Nothing downstream — calibration, grounding, the Phase 1 promotion — can proceed without evidence from a run that actually happened.
2	Add a licence column to the competitive table	Hours	The two models named as strongest cannot be shipped. Recording that now prevents a plan being built on them.
3	Fix the character-error-rate reporting	Hours	The OCR stage scores 6.6% error strictly and 0.0% ignoring spacing — it reads \$1,042.60 as \$ 1 , 042.60. That is a spacing convention, not a misreading, and the headline number currently overstates the error.
4	Run two measurements borrowed from these references	Days	A compression sweep for the tier-2 model, and one serving-layer comparison for the OCR service. Both are gaps the plan does not currently close; both have an existing harness.

The honest summary

The engine's design is well ahead of both reference projects and, on the dimensions an accounting customer actually buys, well ahead of the open models too: provenance, calibrated abstention, deterministic validation, replay, audit trail, residency as a type. None of that is aspiration — most of it is built and tested.

What the references expose is narrower and fairly uncomfortable: **two small projects with almost no code have published measurements we have not taken**. One measured its serving stack; one measured its model compression. We have measured neither, on a system far larger than both, whose cost model assumes answers to exactly those two questions.

The fix is not more architecture. The architecture is good. The fix is to switch on the stage that is already written, run it against real documents, and fill in the numbers our own code marks as `null`.

Sources. Snap Apps repository as of 12 September 2026: `docs/OCR.md`, `packages/docai/src/registry.ts`, `packages/docai/src/pipeline.ts`, `apps/server/src/extraction/shadow.ts`, `apps/server/bench/results/`, `services/docai-engine/`. External: `sentiment.net1io.space`; `github.com/azlanabas/utmcs_llmv`; Hugging Face model index, queried 12 September 2026.

Caveats. Benchmark figures are on four synthetic documents and do not represent accuracy on real captured paperwork. Download counts and licences are point-in-time. The two external projects were read, not benchmarked — no number in this document was produced by running their code.